UNIWERSYTET GDAŃSKI · WMFiI · INFORMATYKA

Testowanie automatyczne

Lab 1 Wprowadzenie do testowania

1 Cel zajęć

Na pierwszych zajęciach nie piszemy jeszcze projektu. Poznajemy mapę terenu: jakie są rodzaje testów, jak się do siebie mają, jakich narzędzi używamy i dlaczego umiejętność testowania jest dziś ważniejsza niż kiedykolwiek.

- **Po zajęciach:** odróżnisz testy funkcjonalne od niefunkcjonalnych i podasz przykłady każdego rodzaju.
- Wytłumaczysz, co to piramida testów i czemu jej kształt ma znaczenie.
- Zorientujesz się, jakie narzędzia pasują do jakiego poziomu testów.
- Zrozumiesz zasady zaliczenia i będziesz gotowy na kolejne laboratoria.

2 Sprawy organizacyjne

- **Godzina rozpoczęcia:** [do uzupełnienia]
- **Wykłady:** [do uzupełnienia]
- **Materiały:** PDF-y z każdych zajęć na stronie kursu (poniżej).
- **Forma zaliczenia:** projekt (szczegóły w sekcji „Zasady zaliczenia”).

Zespół na Microsoft Teams

Ogłoszenia i materiały pojawiają się w zespole kursu. Zapisz się jeszcze dziś.

Dołącz do zespołu na Teams

lub zeskanuj kod QR

Strona kursu

Materiały z zajęć (PDF-y), zadania i informacje dla studentów.

konrad-ug.github.io

lub zeskanuj kod QR

3 Zasady zaliczenia

Zasady mogą ulec zmianie. Terminy oraz kryteria obrony projektów zostaną podane bliżej końca semestru.

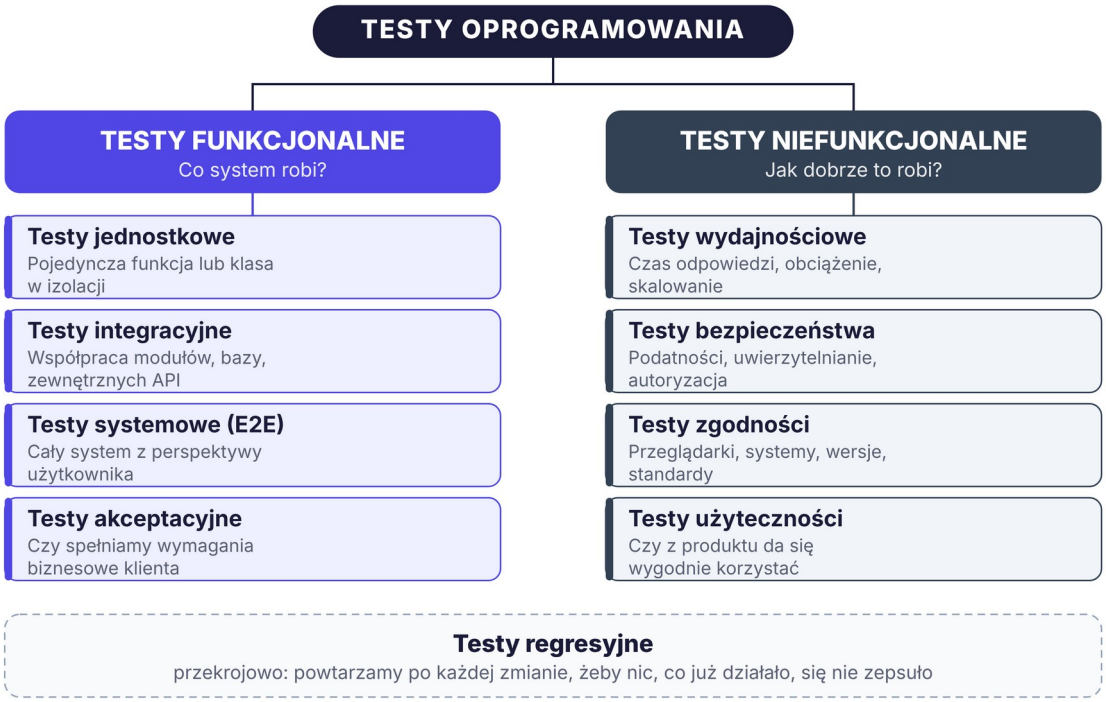
3.0	**Projekt bankowy** Repozytorium z działającym projektem bankowym (wszystkie feature) z testami oraz zielonymi pipeline'ami.
3.5 – 5.0	**Projekt własny** Dodatkowy projekt (wymagania zostaną podane później) z opisem, testami i pipeline'ami.

4 Wymagania wstępne

Język programowania	Python lub inny język programowania
Edytor / IDE	Visual Studio Code
Konto GitHub	Załóż konto na github.com przed zajęciami; podstawowe komendy git, GitHub Actions (znajomość podstaw mile widziana)
API	Postman, znajomość REST API
Język angielski	Dokumentacja i narzędzia są po angielsku
Umiejętności	Umiejętność korzystania z Google

5 Testy funkcjonalne i niefunkcjonalne

Testy dzielimy na dwie wielkie rodziny. Funkcjonalne sprawdzają, co system robi (czy przelew się wykona). Niefunkcjonalne sprawdzają, jak dobrze to robi (czy przelew wykona się szybko, bezpiecznie i na każdym urządzeniu).



6 Poziomy testów i wartość biznesowa

Piramida testów pokazuje, ile testów danego poziomu warto mieć. Im niżej, tym testy są szybsze, tańsze i stabilniejsze, więc piszemy ich najwięcej. Im wyżej, tym bliżej prawdziwego użytkownika, ale też wolniej, drożej i kruchej. Z perspektywy biznesu testy jednostkowe dają szybką informację zwrotną po każdej zmianie, a testy E2E sprawdzają to, na czym naprawdę zależy klientowi. Potrzebujemy obu, w odpowiednich proporcjach.



7 Czy piramida zawsze wygląda tak samo?

Nie. Proporcje testów zależą od projektu: od ryzyka, kosztu błędu i tego, jak łatwo daną rzecz przetestować. Klasyczna piramida jest dobrym punktem wyjścia, a nie prawem natury.

> **Pytanie do grupy**
>
> Czy potrafisz podać przykład projektu, w którym warto napisać wszystkie możliwe testy? Co o tym decyduje?

8 Po co nam testy automatyczne i czy zawsze są nam potrzebne?

Kiedy automatyzacja się opłaca

- Testy są powtarzalne i uruchamiamy je często (np. po każdej zmianie w kodzie).
- Chcemy szybko wykrywać regresje, zanim zobaczy je użytkownik.
- Testy są częścią pipeline'u CI i blokują wadliwy kod przed wdrożeniem.

Kiedy lepiej testować ręcznie

- Funkcja jest eksperymentalna i będzie się szybko zmieniać.
- Sprawdzamy wrażenia użytkownika, np. wygodę i wygląd interfejsu.
- Test uruchomimy raz i koszt automatyzacji nigdy się nie zwróci.

Dlaczego w erze AI testowanie jest ważniejsze

- **Kod powstaje szybciej, niż człowiek jest w stanie go przeczytać.** Wąskim gardłem staje się weryfikacja, nie pisanie.
- **AI myli się przekonująco.** Wygenerowany kod wygląda poprawnie, a potrafi zawierać subtelne błędy, wymyślone API albo luki w bezpieczeństwie.
- **Testy są specyfikacją i siatką bezpieczeństwa.** Jeśli wiesz, co kod ma robić, i masz to w testach, możesz bezpiecznie korzystać z pomocy AI.
- **Odpowiedzialność zostaje po stronie człowieka.** AI może wygenerować testy, ale to Ty oceniasz, czy testują to, co trzeba.

9 Narzędzia

Przegląd narzędzi, z którymi spotkasz się na zajęciach i w pracy. Nie musisz znać ich wszystkich, ważniejsze jest zrozumienie, do którego poziomu piramidy pasują.



Dokumentacja: pytest · JUnit · Jest · Playwright · Cypress · Selenium · Postman · GitHub Actions · Docker · Jenkins

Do poczytania: The Practical Test Pyramid (Martin Fowler)

AI

Treść wygenerowana lub zmodyfikowana z użyciem AI

Materiał powstał z pomocą narzędzi sztucznej inteligencji.
Oznaczenie przejrzystości w duchu unijnego AI Act.